Day02 NumPy核心与向量化计算

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 2 天（8 课时）
> 主题：深入理解 NumPy 设计思想、ndarray 操作、广播机制、向量化性能

一、本日目标

1. 理解 NumPy 的设计思想：为什么需要 ndarray、向量化为什么快；
2. 掌握 ndarray 的创建、属性、索引、切片；
3. 深入理解广播机制，能预判不同 shape 数组的运算结果；
4. 掌握常用数学与统计函数，理解 axis 参数的本质；
5. 掌握矩阵运算与线性代数基础；
6. 通过性能对比实验，理解 Python 循环 vs NumPy 向量化的差异；
7. 项目实战：用向量化实现 Haversine 球面距离，计算 10 万点距离矩阵。

二、知识要点

- NumPy 的核心是 ndarray（N 维数组），底层用 C 实现，内存连续；
- 向量化 = 用数组运算替代 Python 循环，底层 C 执行，速度快 10~100 倍；
- 广播机制让不同 shape 的数组可以运算，不需要手动复制；
- axis 参数指定沿哪个维度操作，是 NumPy 最容易混淆的概念之一；
- 本项目大量使用 NumPy：Haversine 距离、k-means 距离矩阵、统计计算。

课时1：NumPy设计思想

1.1 课时目标

理解"为什么需要 NumPy"、"向量化为什么快"，建立性能意识。

1.2 知识点深度讲解

1.2.1 Python 列表的问题

Python 的列表（list）是一个"指针数组"，每个元素是一个指向 Python 对象的指针。这意味着：
- **内存不连续**：列表元素在内存中分散存储，CPU 缓存命中率低；
- **类型不固定**：列表可以混合不同类型（`[1, "a", 3.14]`），每次操作都要检查类型；
- **循环慢**：Python 循环是解释执行，每次迭代都有类型检查、函数调用等开销。

例如，计算 100 万个数字的平方：

```
# Python 列表方式（慢）
nums = list(range(1000000))
squares = [x ** 2 for x in nums]   # 约 0.1 秒
```

1.2.2 NumPy ndarray 的优势

NumPy 的 ndarray 是一个"同类型、内存连续"的数组：
- **内存连续**：所有元素在内存中连续存储，CPU 缓存命中率高；
- **类型固定**：创建时指定 dtype（如 float64），不需要运行时类型检查；
- **向量化运算**：用 C 实现的底层函数一次性处理整个数组，不需要 Python 循环。

同样计算 100 万个数字的平方：

```
# NumPy 方式（快）
import numpy as np
nums = np.arange(1000000)
squares = nums ** 2   # 约 0.003 秒，快 30 倍
```

1.2.3 向量化的本质

向量化（Vectorization）= 把"对每个元素做操作"的 Python 循环，替换成"对整个数组做操作"的 NumPy 函数。

原理：
1. NumPy 函数底层用 C/Fortran 实现，直接操作连续内存；
2. 不需要 Python 解释器的逐行解释开销；
3. 可以利用 CPU 的 SIMD（单指令多数据）指令，一次处理多个数据；
4. 多线程/多进程加速（部分函数支持）。

> 本项目中，53 万行数据的距离计算、k-means 聚类、统计分析，如果用 Python 循环可能需要几

> 分钟甚至几十分钟，用 NumPy 向量化只需要几秒。

1.2.4 NumPy 在本项目中的应用场景

场景	NumPy 操作	替代的 Python 循环
Haversine 距离	数组运算 + np.sin/np.cos	双重循环算每对点距离
k-means 距离矩阵	广播 + 向量化距离	循环算每个点到每个中心的距离
统计计算	np.mean/np.std/np.sum	循环累加
条件过滤	布尔索引	if 判断 + append
排序	np.argsort	手写排序算法

1.3 项目实现步骤

本课时不直接操作项目数据，通过小实验理解 NumPy 的性能优势。

1.4 完整代码（性能对比实验）

```
# ============================================================
# 课时1：NumPy设计思想 - 性能对比实验
# 功能：对比 Python 循环 vs NumPy 向量化的速度差异
# ============================================================

import time          # 计时
import numpy as np   # 导入numpy

N = 1000000   # 100万个数据

# ------------------------------------------------------------
# 实验1：计算平方
# ------------------------------------------------------------
print("=" * 50)
print("实验1：计算 100 万个数字的平方")
print("=" * 50)

# Python 列表方式
py_list = list(range(N))
t0 = time.time()
py_result = [x ** 2 for x in py_list]   # 列表推导式（比for循环快，但仍是Python循环）
t_py = time.time() - t0
print(f"Python 列表推导式: {t_py:.4f} 秒")

# NumPy 方式
np_arr = np.arange(N)
t0 = time.time()
np_result = np_arr ** 2   # 向量化运算
t_np = time.time() - t0
print(f"NumPy 向量化:      {t_np:.4f} 秒")
print(f"加速比: {t_py / t_np:.1f} 倍")
print()

# ------------------------------------------------------------
# 实验2：计算求和
# ------------------------------------------------------------
```

```
print("=" * 50)
print("实验2：计算 100 万个数字的和")
print("=" * 50)

# Python 方式
t0 = time.time()
py_sum = sum(py_list)   # Python 内置 sum（C实现，但仍需遍历Python对象）
t_py = time.time() - t0
print(f"Python sum():  {t_py:.4f} 秒，结果={py_sum}")

# NumPy 方式
t0 = time.time()
np_sum = np_arr.sum()   # NumPy sum（C实现，连续内存）
t_np = time.time() - t0
print(f"NumPy .sum():  {t_np:.4f} 秒，结果={np_sum}")
print(f"加速比：{t_py / t_np:.1f} 倍")
print()

# ------------------------------------------------------------
# 实验3：矩阵乘法（最能体现向量化优势）
# ------------------------------------------------------------
print("=" * 50)
print("实验3：1000x1000 矩阵乘法")
print("=" * 50)

A = np.random.rand(1000, 1000)   # 1000x1000 随机矩阵
B = np.random.rand(1000, 1000)

# NumPy 矩阵乘法（底层调用 BLAS，高度优化）
t0 = time.time()
C = A @ B   # 等价于 np.matmul(A, B)
t_np = time.time() - t0
print(f"NumPy 矩阵乘法：{t_np:.4f} 秒")
print("（Python 三重循环实现需要几十分钟，不做对比了）")
print()

print("=" * 50)
```

```
print("结论：数据量越大、运算越复杂，NumPy 向量化的优势越明显")
print("=" * 50)
```

1.5 运行结果与解读

运行后应看到：
- 平方计算：NumPy 比 Python 快约 20~50 倍；
- 求和：NumPy 比 Python 快约 5~20 倍；
- 矩阵乘法：NumPy 不到 1 秒，Python 三重循环需要几十分钟。

> 注意：具体加速比因电脑性能而异，但趋势一致——数据量越大、运算越复杂，NumPy 优势越明显。

1.6 当堂练习

1. 把 N 改成 1000 万，重新运行实验1，观察加速比变化；
2. 用 Python 循环实现两个 100x100 矩阵的乘法，计时，再用 NumPy 实现，对比加速比；
3. 思考：为什么 NumPy 的 `sum` 比 Python 的 `sum` 快？（提示：内存连续性 + 类型固定）

1.7 课时小结

- Python 列表内存不连续、类型不固定、循环慢；
- NumPy ndarray 内存连续、类型固定、向量化运算快；
- 向量化 = 用 C 实现的数组函数替代 Python 循环；
- 数据量越大，NumPy 优势越明显。

课时2：ndarray创建与属性

2.1 课时目标

掌握 ndarray 的多种创建方式和核心属性。

2.2 知识点深度讲解

2.2.1 什么是 ndarray

ndarray（N-dimensional array）是 NumPy 的核心数据结构，表示一个 N 维数组。关键特征：
- **同类型**：所有元素必须是同一类型（dtype）；

- **固定大小**：创建后大小（shape）固定，不能像列表一样 append；
- **内存连续**：数据在内存中连续存储；
- **支持向量化**：可以对整个数组做运算。

2.2.2 创建 ndarray 的方式

方式1：从 Python 列表转换

```
import numpy as np

# 一维数组
a = np.array([1, 2, 3, 4, 5])
print(a)          # [1 2 3 4 5]

# 二维数组（矩阵）
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b)
# [[1 2 3]
#  [4 5 6]]

# 指定数据类型
c = np.array([1, 2, 3], dtype=np.float64)
print(c.dtype)    # float64
```

方式2：用函数创建特殊数组

```
# 全0数组
np.zeros(5)              # [0. 0. 0. 0. 0.]
np.zeros((2, 3))         # 2行3列全0

# 全1数组
np.ones(5)               # [1. 1. 1. 1. 1.]
np.ones((2, 3))          # 2行3列全1

# 填充指定值
np.full(5, 3.14)         # [3.14 3.14 3.14 3.14 3.14]
np.full((2, 2), 7)       # 2x2 全7

# 单位矩阵
np.eye(3)                 # 3x3 单位矩阵
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

# 空数组（不初始化，值随机，最快）
np.empty(5)               # 5个未初始化的值
```

方式3：用序列函数创建

```
# arange：类似 range，但返回 ndarray
np.arange(5)              # [0 1 2 3 4]
np.arange(1, 10, 2)       # [1 3 5 7 9]（起始，终止(不含)，步长）

# linspace：等差数列（指定元素个数）
np.linspace(0, 1, 5)      # [0.   0.25 0.5  0.75 1.  ]（5个等间距点）

# logspace：等比数列
np.logspace(0, 3, 4)      # [1. 10. 100. 1000.]（10^0 到 10^3，4个点）
```

方式4：用随机函数创建

```
# 均匀分布 [0, 1)
np.random.rand(5)              # 5个 [0,1) 随机数
np.random.rand(2, 3)           # 2x3 随机矩阵

# 标准正态分布（均值0，方差1）
np.random.randn(5)             # 5个标准正态随机数
np.random.randn(2, 3)          # 2x3 标准正态矩阵

# 指定范围的随机整数
np.random.randint(0, 10, 5)    # 5个 [0,10) 随机整数

# 设置随机种子（可复现）
np.random.seed(42)              # 设置种子后，随机数序列固定
np.random.rand(3)               # 每次运行结果相同
```

> 本项目中，`np.random.default_rng(seed)` 是新版随机数生成器，比 `np.random.seed()` 更推荐。k-means 的初始中心选择用这个。

2.2.3 ndarray 的核心属性

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float64)

print(a.shape)       # (2, 3) — 形状：2行3列
print(a.ndim)        # 2 — 维度数（秩）
print(a.size)        # 6 — 元素总数（2*3）
print(a.dtype)       # float64 — 数据类型
print(a.itemsize)    # 8 — 每个元素的字节数（float64=8字节）
print(a.nbytes)      # 48 — 总字节数（6*8）
print(a.strides)     # (24, 8) — 每个维度步进的字节数
```

属性详解：
- **shape**：元组，表示每个维度的大小。`(2, 3)` 表示 2 行 3 列；
- **ndim**：维度数，等于 `len(shape)`；
- **size**：元素总数，等于 `np.prod(shape)`；
- **dtype**：数据类型，如 int32、float64、bool、`<U10`（字符串）；

- **itemsize**：每个元素的字节数；
- **nbytes**：总字节数 = size × itemsize；
- **strides**：每个维度步进的字节数，理解内存布局的关键。

> 本项目中，经纬度数据通常是 `float64`（8字节），imsi 是字符串类型。理解 dtype 很重要，因为不同 dtype 占用内存不同，运算速度也不同。

2.2.4 数据类型（dtype）

常用数据类型：
类型	字节	范围	用途
int8	1	-128~127	小整数
int32	4	-21亿~21亿	普通整数
int64	8	很大	大整数
float32	4	约7位有效数字	省内存的浮点数
float64	8	约15位有效数字	标准浮点数（默认）
bool	1	True/False	布尔值
`<U10`	40	10个Unicode字符	字符串

> 本项目中，经纬度用 float64（精度足够），标签用 int8（0/1，省内存），imsi 用字符串。

2.3 项目实现步骤

1. 用不同方式创建 ndarray；
2. 查看各属性；
3. 理解 dtype 对内存的影响。

2.4 完整代码（ndarray 创建与属性练习）

```
# ============================================================
# 课时2：ndarray 创建与属性
# ============================================================

import numpy as np

# ------------------------------------------------------------
# 1. 多种创建方式
# ------------------------------------------------------------
print("=== 1. 从列表转换 ===")
a1 = np.array([1, 2, 3, 4, 5])
print(f"一维：{a1}，shape={a1.shape}")

a2 = np.array([[1, 2, 3], [4, 5, 6]])
print(f"二维:\n{a2}，shape={a2.shape}")

print("\n=== 2. 特殊数组 ===")
print(f"zeros(5)：{np.zeros(5)}")
print(f"ones((2,3)):\n{np.ones((2, 3))}")
print(f"full(3, 7)：{np.full(3, 7)}")
print(f"eye(3):\n{np.eye(3)}")

print("\n=== 3. 序列函数 ===")
print(f"arange(1, 10, 2)：{np.arange(1, 10, 2)}")
print(f"linspace(0, 1, 5)：{np.linspace(0, 1, 5)}")

print("\n=== 4. 随机数组（设置种子可复现） ===")
rng = np.random.default_rng(42)   # 新版随机数生成器
print(f"均匀分布(5个)：{rng.random(5)}")
print(f"标准正态(5个)：{rng.standard_normal(5)}")
print(f"随机整数[0,10) 5个：{rng.integers(0, 10, 5)}")

# ------------------------------------------------------------
# 2. 核心属性
# ------------------------------------------------------------
print("\n=== 5. 核心属性 ===")
```

```
a = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]], dtype=np.float64)
print(f"shape:    {a.shape}    (形状：2行3列)")
print(f"ndim:     {a.ndim}        (维度数)")
print(f"size:     {a.size}        (元素总数=2*3)")
print(f"dtype:    {a.dtype}    (数据类型)")
print(f"itemsize: {a.itemsize}        (每元素字节数)")
print(f"nbytes:   {a.nbytes}       (总字节数=6*8)")
print(f"strides:  {a.strides}    (每维度步进字节数)")

# ------------------------------------------------------------
# 3. dtype 对内存的影响
# ------------------------------------------------------------
print("\n=== 6. dtype 内存对比 ===")
N = 1000000
arr_f64 = np.ones(N, dtype=np.float64)
arr_f32 = np.ones(N, dtype=np.float32)
arr_i8 = np.ones(N, dtype=np.int8)
print(f"float64: {arr_f64.nbytes / 1024 / 1024:.1f} MB")
print(f"float32: {arr_f32.nbytes / 1024 / 1024:.1f} MB (省一半)")
print(f"int8:    {arr_i8.nbytes / 1024 / 1024:.1f} MB (省87.5%)")
print("→ 本项目中，标签(0/1)用int8，经纬度用float64，imsi用字符串")
```

2.5 运行结果与解读

运行后展示各种创建方式、属性值、dtype 内存对比。特别注意：
- 100 万个 float64 占 8MB，float32 占 4MB，int8 只占 1MB；
- 合理选择 dtype 可以大幅节省内存，本项目 53 万行数据如果全用 float64 也只占几十 MB，内存不是问题，但养成好习惯。

2.6 当堂练习

1. 用 `np.linspace` 创建 0 到 2π 之间的 100 个等间距点，然后计算 `np.sin(x)`；
2. 创建一个 1000x1000 的随机矩阵，分别用 float64 和 float32，比较内存占用；
3. 用 `np.random.default_rng(0)` 创建一个 3x3 的标准正态随机矩阵，打印其 shape、dtype、nbytes。

2.7 课时小结

- ndarray 创建：从列表转换、zeros/ones/full、arange/linspace、随机函数；

- 核心属性：shape、ndim、size、dtype、itemsize、nbytes、strides；
- 合理选择 dtype 节省内存；
- `np.random.default_rng(seed)` 是新版随机数生成器，推荐使用。

课时3：索引与切片

3.1 课时目标

掌握 ndarray 的一维/二维索引、布尔索引、花式索引，理解视图与副本的区别。

3.2 知识点深度讲解

3.2.1 一维索引与切片

一维数组的索引和切片与 Python 列表类似：

```
import numpy as np

a = np.arange(10)   # [0 1 2 3 4 5 6 7 8 9]

# 索引
a[0]      # 0（第一个元素）
a[-1]     # 9（最后一个元素）
a[3]      # 3

# 切片 [起始:终止:步长]（含起始，不含终止）
a[2:5]    # [2 3 4]
a[:5]     # [0 1 2 3 4]（起始省略，从0开始）
a[5:]     # [5 6 7 8 9]（终止省略，到末尾）
a[::2]    # [0 2 4 6 8]（步长2，隔一个取一个）
a[::-1]   # [9 8 7 6 5 4 3 2 1 0]（步长-1，反转）
```

3.2.2 二维索引与切片

二维数组的索引用 `[行, 列]`：

```
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])

# 单个元素
a[0, 0]      # 1（第0行第0列）
a[1, 2]      # 7（第1行第2列）
a[-1, -1]    # 12（最后一行最后一列）

# 整行/整列
a[0]         # [1 2 3 4]（第0行，等价于 a[0, :]）
a[:, 0]      # [1 5 9]（第0列）

# 切片
a[0:2, 1:3]  # [[2 3]
             #  [6 7]]（前2行，第1-2列）
a[:2, :]     # 前2行所有列
a[:, 1:]     # 所有行，第1列之后
```

> 注意：`a[0][1]` 和 `a[0, 1]` 结果相同，但 `a[0, 1]` 更高效（一次索引操作），推荐用逗号分隔的方式。

3.2.3 布尔索引

布尔索引用一个布尔数组选择元素，是 NumPy 最强大的特性之一：

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# 条件判断返回布尔数组
a > 5        # [False False False False False  True  True  True  True  True]

# 用布尔数组索引（选出满足条件的元素）
a[a > 5]     # [ 6  7  8  9 10]

# 多条件组合（& = 且，| = 或，~ = 非）
a[(a > 3) & (a < 8)]   # [4 5 6 7]（大于3且小于8）
a[(a % 2 == 0) | (a > 7)]  # [2 4 6 8 9 10]（偶数或大于7）

# 用布尔索引修改元素
a[a > 5] = 0   # 把大于5的元素设为0
print(a)        # [1 2 3 4 5 0 0 0 0 0]
```

> 本项目中，布尔索引大量使用：过滤出行点（`label == 1`）、过滤驻留点（`label == 0`）、过滤低频点（`freq < 4`）、过滤目标日期（`time.startswith(...)`）等。

3.2.4 花式索引（Fancy Indexing）

花式索引用整数数组选择元素：

```
a = np.arange(10)   # [0 1 2 3 4 5 6 7 8 9]

# 用整数数组选择
a[[0, 2, 4, 6]]    # [0 2 4 6]（选第0,2,4,6个元素）
a[[1, 1, 2, 3]]    # [1 1 2 3]（可以重复选择）

# 二维花式索引
b = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
b[[0, 2]]           # 选第0行和第2行
b[[0, 2], [1, 2]]   # 选 (0,1) 和 (2,2) 两个元素 → [2 9]
```

3.2.5 视图（View）vs 副本（Copy）

这是 NumPy 最容易踩的坑之一：

```
a = np.array([1, 2, 3, 4, 5])

# 切片返回的是视图（共享内存）
b = a[1:4]      # b 是 a 的视图
b[0] = 99       # 修改 b
print(a)        # [ 1 99  3  4  5] — a 也被修改了！

# 花式索引和布尔索引返回的是副本（不共享内存）
c = a[[0, 2, 4]]   # c 是副本
c[0] = 88          # 修改 c
print(a)           # [ 1 99  3  4  5] — a 不受影响

# 显式复制
d = a[1:4].copy()   # 显式复制
d[0] = 77
print(a)           # [ 1 99  3  4  5] — a 不受影响
```

> 规则：**切片（含整数索引）返回视图，花式索引和布尔索引返回副本**。修改切片结果会影响原数组，这是常见的 bug 来源。如果不想影响原数组，用 `.copy()` 显式复制。
>
> 本项目中，`df = cleaned[cleaned["label"] == 0]` 返回的是副本（布尔索引），修改 df 不会影响 cleaned。但 `df = cleaned.iloc[1:10]` 返回的是视图，修改 df 可能影响 cleaned（会有 SettingWithCopyWarning）。

3.3 项目实现步骤

1. 练习一维/二维索引切片；
2. 练习布尔索引过滤数据；
3. 理解视图与副本的区别。

3.4 完整代码（索引与切片练习）

```
# ============================================================
# 课时3：索引与切片
# ============================================================

import numpy as np

# ------------------------------------------------------------
# 1. 一维索引与切片
# ------------------------------------------------------------
print("=== 1. 一维索引与切片 ===")
a = np.arange(10)
print(f"原数组：{a}")
print(f"a[0]: {a[0]}, a[-1]: {a[-1]}")
print(f"a[2:5]: {a[2:5]}")
print(f"a[::2]: {a[::2]}")
print(f"a[::-1]: {a[::-1]}（反转）")

# ------------------------------------------------------------
# 2. 二维索引与切片
# ------------------------------------------------------------
print("\n=== 2. 二维索引与切片 ===")
b = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print(f"原数组:\n{b}")
print(f"b[0, 0] = {b[0, 0]}")
print(f"b[1, 2] = {b[1, 2]}")
print(f"第0行：{b[0]}")
print(f"第0列：{b[:, 0]}")
print(f"前2行第1-2列:\n{b[0:2, 1:3]}")

# ------------------------------------------------------------
# 3. 布尔索引（本项目最常用）
# ------------------------------------------------------------
print("\n=== 3. 布尔索引 ===")
c = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(f"原数组：{c}")
print(f"c > 5 的布尔数组：{c > 5}")
```

```
print(f"c[c > 5] = {c[c > 5]}（大于5的元素）")
print(f"c[(c > 3) & (c < 8)] = {c[(c > 3) & (c < 8)]}（大于3且小于8）")
print(f"c[c % 2 == 0] = {c[c % 2 == 0]}（偶数）")

# 模拟项目场景：过滤出行点
labels = np.array([0, 1, 0, 0, 1, 1, 0, 1, 0, 0])
lons = np.array([123.41, 123.42, 123.43, 123.44, 123.45,
                 123.46, 123.47, 123.48, 123.49, 123.50])
travel_lons = lons[labels == 1]   # 过滤出行点的经度
print(f"\n模拟项目：所有经度：{lons}")
print(f"标签：{labels} (0=驻留, 1=出行)")
print(f"出行点经度：{travel_lons}")

# ------------------------------------------------------------
# 4. 视图 vs 副本（重要！）
# ------------------------------------------------------------
print("\n=== 4. 视图 vs 副本 ===")
d = np.array([1, 2, 3, 4, 5])
print(f"原数组 d: {d}")

# 切片 = 视图
d_view = d[1:4]
d_view[0] = 99
print(f"切片修改后 d: {d}（被修改了！因为切片是视图）")

# 布尔索引 = 副本
d2 = np.array([1, 2, 3, 4, 5])
d_copy = d2[d2 > 2]
d_copy[0] = 88
print(f"布尔索引修改后 d2: {d2}（未被修改，因为布尔索引是副本）")

# 显式复制
d3 = np.array([1, 2, 3, 4, 5])
d_safe = d3[1:4].copy()
d_safe[0] = 77
print(f"显式复制修改后 d3: {d3}（未被修改）")
```

3.5 运行结果与解读

运行后展示各种索引方式，特别注意视图 vs 副本的区别：
- 切片修改会影响原数组（视图）；
- 布尔索引修改不影响原数组（副本）；
- 用 `.copy()` 显式复制可以避免意外修改。

3.6 当堂练习

1. 创建一个 5x5 的随机矩阵，提取第 2 行第 3 列的元素，以及第 2-4 行第 1-3 列的子矩阵；
2. 用布尔索引从 `np.arange(100)` 中选出所有能被 3 整除的数；
3. 思考：`a = np.array([1,2,3]); b = a; b[0] = 99`，此时 `a[0]` 是多少？为什么？（提示：NumPy 数组的赋值是引用，不是复制）

3.7 课时小结

- 一维索引切片与 Python 列表类似；
- 二维索引用 `[行, 列]`，推荐逗号分隔；
- 布尔索引是 NumPy 最强大的特性，本项目大量使用；
- 切片返回视图（共享内存），布尔/花式索引返回副本；
- 用 `.copy()` 显式复制，避免意外修改原数组。

课时4：广播机制

4.1 课时目标

深入理解广播机制，能预判不同 shape 数组的运算结果。

4.2 知识点深度讲解

4.2.1 什么是广播

广播（Broadcasting）是 NumPy 对不同 shape 的数组进行运算的机制。当两个数组 shape 不同时，NumPy 会自动把小的数组"广播"成大数组的 shape，使它们可以运算。

最简单的例子：数组 + 标量

```
import numpy as np

a = np.array([1, 2, 3])
b = a + 1      # 标量 1 被广播成 [1, 1, 1]
print(b)       # [2 3 4]
```

这里，标量 1 被"广播"成了和 a 相同 shape 的数组 `[1, 1, 1]`，然后逐元素相加。

4.2.2 广播规则

NumPy 的广播规则（从右往左比较维度）：

1. 如果两个数组的维度数不同，在维度少的数组前面补 1；
2. 从右往左比较每个维度：
   - 如果维度大小相同，兼容；
   - 如果其中一个维度大小是 1，兼容（会被广播成另一个的大小）；
   - 否则不兼容，报错。

示例1：二维数组 + 一维数组

```
a = np.ones((3, 4))    # shape (3, 4)
b = np.arange(4)        # shape (4,)

# 广播过程：
# b 的 shape (4,) → 补维度 → (1, 4)
# 比较：(3, 4) vs (1, 4) → 第1维1→广播成3，第2维相同→兼容
# 结果 shape (3, 4)
c = a + b
print(c.shape)   # (3, 4)
print(c)
# [[1. 2. 3. 4.]
#  [1. 2. 3. 4.]
#  [1. 2. 3. 4.]]
```

示例2：列向量 + 行向量

```
a = np.array([[1], [2], [3]])    # shape (3, 1) — 列向量
b = np.array([10, 20, 30, 40])   # shape (4,) — 行向量

# 广播过程：
# a: (3, 1)
# b: (4,) → 补维度 → (1, 4)
# 比较：(3, 1) vs (1, 4) → 第1维1→广播成3，第2维1→广播成4
# 结果 shape (3, 4)
c = a + b
print(c)
# [[11 21 31 41]
#  [12 22 32 42]
#  [13 23 33 43]]
```

> 这就是"外积"的广播实现，非常有用。

示例3：不兼容的广播（报错）

```
a = np.ones((3, 4))
b = np.ones((2, 4))

# 比较：(3, 4) vs (2, 4) → 第1维 3 vs 2，都不是1 → 不兼容
# c = a + b  # 报错：ValueError: operands could not be broadcast together
```

4.2.3 广播的本质

广播不会真正复制数组，而是在运算时"虚拟"地扩展。这意味着：
- 广播运算内存效率高（不需要实际复制数据）；
- 但如果需要保存广播后的结果，会分配新内存。

4.2.4 广播在本项目中的应用

本项目中广播机制无处不在：

应用1：经纬度数组 + 偏移量

```
lons = np.array([123.41, 123.42, 123.43])   # shape (3,)
lats = np.array([41.80, 41.81, 41.82])       # shape (3,)

# 每个经纬度都加一个偏移量（标量广播）
lons_shifted = lons + 0.01   # shape (3,)
```

应用2：k-means 距离矩阵（核心）

```
# X: (n, 2) 的点集，centers: (k, 2) 的簇中心
X = np.random.rand(100, 2)       # 100个点，shape (100, 2)
centers = np.random.rand(3, 2)   # 3个中心，shape (3, 2)

# 计算每个点到每个中心的距离（广播实现）
# X[:, None, :] → shape (100, 1, 2)
# centers[None, :, :] → shape (1, 3, 2)
# 广播后 → (100, 3, 2)
diff = X[:, None, :] - centers[None, :, :]   # shape (100, 3, 2)
dist_sq = (diff ** 2).sum(axis=2)             # shape (100, 3)，距离平方
```

> 这是 k-means 中计算距离矩阵的标准写法，用广播替代双重循环，速度快 100 倍以上。

应用3：标准化（均值方差归一化）

```
data = np.random.rand(100, 3)   # 100个样本，3个特征

# 按列标准化（每列减去均值，除以标准差）
mean = data.mean(axis=0)         # shape (3,)，每列均值
std = data.std(axis=0)           # shape (3,)，每列标准差
normalized = (data - mean) / std # data (100,3) - mean (3,) → 广播 → (100,3)
```

4.3 项目实现步骤

1. 理解广播规则；
2. 练习不同 shape 数组的广播；
3. 用广播实现 k-means 距离矩阵。

4.4 完整代码（广播机制练习）

```
# ============================================================
# 课时4：广播机制
# ============================================================

import numpy as np

# ------------------------------------------------------------
# 1. 基础广播：数组 + 标量
# ------------------------------------------------------------
print("=== 1. 数组 + 标量 ===")
a = np.array([1, 2, 3])
print(f"a = {a}")
print(f"a + 1 = {a + 1}（标量1被广播成[1,1,1]）")

# ------------------------------------------------------------
# 2. 二维 + 一维
# ------------------------------------------------------------
print("\n=== 2. 二维(3,4) + 一维(4,) ===")
a = np.ones((3, 4))
b = np.arange(4)
print(f"a shape: {a.shape}, b shape: {b.shape}")
c = a + b
print(f"结果 shape: {c.shape}")
print(c)

# ------------------------------------------------------------
# 3. 列向量 + 行向量（外积）
# ------------------------------------------------------------
print("\n=== 3. 列向量(3,1) + 行向量(4,) ===")
a = np.array([[1], [2], [3]])      # (3, 1)
b = np.array([10, 20, 30, 40])     # (4,)
c = a + b
print(f"结果 shape: {c.shape}")
print(c)

# ------------------------------------------------------------
```

```
# 4. 广播规则验证
# ------------------------------------------------------------
print("\n=== 4. 广播规则验证 ===")
print("规则：从右往左比较，维度相同或其中一个为1则兼容")

# 兼容的例子
x = np.ones((2, 3, 4))
y = np.ones((3, 4))
z = x + y   # y → (1,3,4) → 广播 → (2,3,4)
print(f"(2,3,4) + (3,4) → 结果 shape: {z.shape} ✓")

# 不兼容的例子
try:
    x = np.ones((3, 4))
    y = np.ones((2, 4))
    z = x + y
except ValueError as e:
    print(f"(3,4) + (2,4) → 报错: {e} ✗")

# ------------------------------------------------------------
# 5. 项目应用：k-means 距离矩阵
# ------------------------------------------------------------
print("\n=== 5. 项目应用：k-means 距离矩阵 ===")
np.random.seed(42)
X = np.random.rand(100, 2)        # 100个点，shape (100, 2)
centers = np.random.rand(3, 2)    # 3个中心，shape (3, 2)

print(f"X shape: {X.shape}")
print(f"centers shape: {centers.shape}")

# 广播计算距离矩阵
X_expanded = X[:, None, :]        # (100, 1, 2)
C_expanded = centers[None, :, :]  # (1, 3, 2)
diff = X_expanded - C_expanded     # 广播 → (100, 3, 2)
dist_sq = (diff ** 2).sum(axis=2) # (100, 3)，每个点到每个中心的距离平方

print(f"距离矩阵 shape: {dist_sq.shape}")
print(f"第0个点到3个中心的距离平方: {dist_sq[0]}")
```

```
print(f"第0个点最近的中心：{dist_sq[0].argmin()}")

# 对比：用双重循环实现（慢）
print("\n对比：双重循环 vs 广播")
import time
t0 = time.time()
dist_loop = np.zeros((100, 3))
for i in range(100):
    for j in range(3):
        dist_loop[i, j] = np.sum((X[i] - centers[j]) ** 2)
t_loop = time.time() - t0

t0 = time.time()
dist_broadcast = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)
t_broadcast = time.time() - t0

print(f"双重循环：{t_loop:.6f} 秒")
print(f"广播实现：{t_broadcast:.6f} 秒")
print(f"加速比：{t_loop / t_broadcast:.1f} 倍")
print(f"结果一致：{np.allclose(dist_loop, dist_broadcast)}")
```

4.5 运行结果与解读

运行后展示各种广播场景，特别注意：
- k-means 距离矩阵用广播实现比双重循环快几十到上百倍；
- 广播规则：从右往左比较，维度相同或其中一个为 1 则兼容；
- `X[:, None, :]` 是增加维度的常用写法（等价于 `X[:, np.newaxis, :]`）。

4.6 当堂练习

1. 预测以下运算的结果 shape：
   - `np.ones((5, 3)) + np.ones((3,))` → ?
   - `np.ones((5, 1)) + np.ones((1, 4))` → ?
   - `np.ones((2, 3, 4)) + np.ones((4,))` → ?
2. 用广播实现两个数组的"差的绝对值"：`a` shape (10, 3)，`b` shape (3,)，结果 shape (10, 3)；
3. 思考：为什么 `np.ones((3, 4)) + np.ones((4, 3))` 会报错？（提示：从右往左比较，第1维 3 vs 4 不兼容）

4.7 课时小结

- 广播让不同 shape 的数组可以运算，不需要手动复制；
- 规则：从右往左比较，维度相同或其中一个为 1 则兼容；
- 广播不实际复制数据，内存效率高；
- 本项目核心应用：k-means 距离矩阵、标准化、数组+标量；
- `X[:, None, :]` 是增加维度的常用写法。

课时5：常用数学与统计函数

5.1 课时目标

掌握 NumPy 常用数学函数和统计函数，深入理解 axis 参数。

5.2 知识点深度讲解

5.2.1 数学函数（逐元素运算）

NumPy 提供大量数学函数，都是**逐元素**（element-wise）运算，返回同 shape 的数组：

```
import numpy as np

x = np.array([0, np.pi/2, np.pi])   # [0, π/2, π]

# 三角函数
np.sin(x)      # [0, 1, 0]（正弦）
np.cos(x)      # [1, 0, -1]（余弦）
np.tan(x)      # [0, inf, 0]（正切，π/2处无穷大）

# 反三角函数
np.arcsin([0, 1])   # [0, π/2]
np.arccos([1, 0])   # [0, π/2]

# 指数与对数
np.exp([0, 1, 2])   # [1, e, e^2]（自然指数）
np.log([1, np.e, np.e**2])   # [0, 1, 2]（自然对数）
np.log10([1, 10, 100])       # [0, 1, 2]（以10为底）

# 幂与开方
np.sqrt([1, 4, 9])    # [1, 2, 3]（平方根）
np.power([2, 3, 4], 2)   # [4, 9, 16]（幂运算）

# 取整
np.floor([1.2, 2.7, -1.2])   # [1, 2, -2]（向下取整）
np.ceil([1.2, 2.7, -1.2])    # [2, 3, -1]（向上取整）
np.round([1.2, 2.7, 1.5])    # [1, 3, 2]（四舍五入）

# 绝对值
np.abs([-1, 2, -3])    # [1, 2, 3]

# 符号
np.sign([-1, 0, 3])    # [-1, 0, 1]（负数→-1，零→0，正数→1）

# 裁剪（限制范围）
np.clip([1, 5, 10, 15], 3, 12)   # [3, 5, 10, 12]（小于3变3，大于12变12）
```

> 本项目中，`np.sin`、`np.cos`、`np.arcsin`、`np.sqrt` 是 Haversine 球面距离公式的核心函数。

5.2.2 统计函数（聚合运算）

统计函数把数组聚合成一个或少数几个值：

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])

# 基本统计
np.sum(a)       # 21（所有元素之和）
np.mean(a)      # 3.5（平均值）
np.std(a)       # 1.7078...（标准差，总体标准差，除以N）
np.var(a)       # 2.9167...（方差）
np.min(a)       # 1（最小值）
np.max(a)       # 6（最大值）
np.argmin(a)    # 0（最小值的索引，扁平化后的索引）
np.argmax(a)    # 5（最大值的索引）

# 累积运算
np.cumsum(a)    # [1, 3, 6, 10, 15, 21]（累积和）
np.cumprod(a)   # [1, 2, 6, 24, 120, 720]（累积积）

# 百分位数
np.percentile(a, 50)   # 3.5（中位数，第50百分位）
np.percentile(a, 25)   # 2.25（第25百分位）

# 唯一值
np.unique([1, 2, 2, 3, 3, 3])   # [1, 2, 3]
```

5.2.3 axis 参数的本质（重点！）

axis 是 NumPy 最容易混淆的概念之一。理解 axis 的关键：

axis 指定"沿着哪个维度做聚合"，那个维度会被"压缩"掉。

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])
# shape (2, 3): axis=0 是行（2行），axis=1 是列（3列）

# 不指定 axis：所有元素聚合，返回标量
np.sum(a)          # 21（所有6个元素之和）

# axis=0：沿着行方向聚合（跨行），行维度被压缩，结果 shape (3,)
np.sum(a, axis=0)  # [5, 7, 9]（每列之和：1+4, 2+5, 3+6）

# axis=1：沿着列方向聚合（跨列），列维度被压缩，结果 shape (2,)
np.sum(a, axis=1)  # [6, 15]（每行之和：1+2+3, 4+5+6）
```

记忆口诀：
- `axis=0`：跨行操作，结果的行数变少（最终变成1行）；
- `axis=1`：跨列操作，结果的列数变少（最终变成1列）。

三维数组的 axis：

```
b = np.ones((2, 3, 4))   # shape (2, 3, 4)
# axis=0：第1个维度（2）被压缩 → 结果 shape (3, 4)
# axis=1：第2个维度（3）被压缩 → 结果 shape (2, 4)
# axis=2：第3个维度（4）被压缩 → 结果 shape (2, 3)

np.sum(b, axis=0).shape   # (3, 4)
np.sum(b, axis=1).shape   # (2, 4)
np.sum(b, axis=2).shape   # (2, 3)
```

> 本项目中，k-means 距离矩阵 `(n, k, 2)` 用 `.sum(axis=2)` 把最后一个维度（经纬度差的平方和）压缩，得到 `(n, k)` 的距离平方矩阵。

5.2.4 keepdims 参数

默认情况下，聚合后被压缩的维度会消失。用 `keepdims=True` 可以保留维度（变成1），便于后续广播：

```
a = np.array([[1, 2, 3], [4, 5, 6]])

np.sum(a, axis=1)              # [6, 15]，shape (2,)
np.sum(a, axis=1, keepdims=True)   # [[6], [15]]，shape (2, 1)

# 应用：每行除以行和（归一化）
row_sums = np.sum(a, axis=1, keepdims=True)   # (2, 1)
normalized = a / row_sums                      # 广播 → (2, 3)
print(normalized)
# [[0.1667 0.3333 0.5   ]
#  [0.2667 0.3333 0.4   ]]
```

5.3 项目实现步骤

1. 练习数学函数和统计函数；
2. 深入理解 axis 参数；
3. 用 keepdims 实现行归一化。

5.4 完整代码（数学与统计函数练习）

```
# ============================================================
# 课时5：常用数学与统计函数
# ============================================================

import numpy as np

# ------------------------------------------------------------
# 1. 数学函数
# ------------------------------------------------------------
print("=== 1. 数学函数 ===")
x = np.array([0, np.pi/4, np.pi/2])
print(f"x = {x}")
print(f"sin(x) = {np.sin(x)}")
print(f"cos(x) = {np.cos(x)}")
print(f"exp(x) = {np.exp(x)}")
print(f"sqrt([1,4,9]) = {np.sqrt([1, 4, 9])}")
print(f"abs([-1,2,-3]) = {np.abs([-1, 2, -3])}")
print(f"clip([1,5,10,15], 3, 12) = {np.clip([1, 5, 10, 15], 3, 12)}")

# ------------------------------------------------------------
# 2. 统计函数
# ------------------------------------------------------------
print("\n=== 2. 统计函数 ===")
a = np.array([[1, 2, 3], [4, 5, 6]])
print(f"数组:\n{a}")
print(f"sum = {np.sum(a)}")
print(f"mean = {np.mean(a)}")
print(f"std = {np.std(a):.4f}")
print(f"var = {np.var(a):.4f}")
print(f"min = {np.min(a)}, max = {np.max(a)}")
print(f"argmin = {np.argmin(a)}, argmax = {np.argmax(a)}")
print(f"cumsum = {np.cumsum(a)}")
print(f"unique([1,2,2,3,3,3]) = {np.unique([1,2,2,3,3,3])}")

# ------------------------------------------------------------
# 3. axis 参数（重点）
```

```
# ------------------------------------------------------------
print("\n=== 3. axis 参数 ===")
print(f"数组 shape: {a.shape}（2行3列）")
print(f"不指定axis: sum={np.sum(a)}（标量，所有元素）")
print(f"axis=0（跨行，每列求和）: {np.sum(a, axis=0)}，shape={np.sum(a,
axis=0).shape}")
print(f"axis=1（跨列，每行求和）: {np.sum(a, axis=1)}，shape={np.sum(a,
axis=1).shape}")

# 三维示例
b = np.ones((2, 3, 4))
print(f"\n三维数组 shape: {b.shape}")
print(f"axis=0 结果 shape: {np.sum(b, axis=0).shape}")
print(f"axis=1 结果 shape: {np.sum(b, axis=1).shape}")
print(f"axis=2 结果 shape: {np.sum(b, axis=2).shape}")

# ------------------------------------------------------------
# 4. keepdims 与归一化
# ------------------------------------------------------------
print("\n=== 4. keepdims 与行归一化 ===")
print(f"原数组:\n{a}")
row_sums = np.sum(a, axis=1, keepdims=True)
print(f"行和（keepdims）: shape={row_sums.shape}\n{row_sums}")
normalized = a / row_sums
print(f"行归一化结果:\n{normalized}")
print(f"每行之和: {np.sum(normalized, axis=1)}（都等于1）")

# ------------------------------------------------------------
# 5. 项目应用：统计经纬度范围
# ------------------------------------------------------------
print("\n=== 5. 项目应用：统计经纬度范围 ===")
# 模拟100个点的经纬度
np.random.seed(42)
coords = np.column_stack([
    np.random.uniform(123.3, 123.5, 100),   # 经度
    np.random.uniform(41.7, 41.9, 100)     # 纬度
])
print(f"坐标数组 shape: {coords.shape}（100个点，2列：经度、纬度）")
```

```
print(f"经度范围: [{coords[:, 0].min():.4f}, {coords[:, 0].max():.4f}]")
print(f"纬度范围: [{coords[:, 1].min():.4f}, {coords[:, 1].max():.4f}]")
print(f"经度均值: {coords[:, 0].mean():.4f}, 纬度均值: {coords[:, 1].mean():.4f}")
print(f"经度标准差: {coords[:, 0].std():.4f}, 纬度标准差: {coords[:, 1].std():.4f}")
```

5.5 运行结果与解读

运行后展示数学函数、统计函数、axis 参数、keepdims、项目应用。特别注意：
- `axis=0` 跨行聚合，`axis=1` 跨列聚合；
- `keepdims=True` 保留维度便于广播；
- 项目中统计经纬度范围用 `min/max/mean/std`。

5.6 当堂练习

1. 创建一个 4x5 的随机矩阵，分别计算每行的均值和每列的标准差；
2. 用 `np.percentile` 计算一个数组的四分位数（25%、50%、75%）；
3. 思考：`np.sum(a, axis=0)` 和 `np.sum(a, axis=1)` 哪个结果的元素更多？为什么？

5.7 课时小结

- 数学函数逐元素运算：sin/cos/exp/log/sqrt/abs/clip 等；
- 统计函数聚合运算：sum/mean/std/var/min/max/argmin/argmax 等；
- axis 指定沿哪个维度聚合，那个维度被压缩；
- keepdims=True 保留维度，便于广播；
- 本项目核心：距离矩阵用 `.sum(axis=2)`，经纬度统计用 `min/max/mean/std`。

课时6：矩阵运算与线性代数

6.1 课时目标

掌握 NumPy 的矩阵运算和线性代数基础，理解其在本项目中的应用。

6.2 知识点深度讲解

6.2.1 矩阵乘法

矩阵乘法是线性代数的核心运算，NumPy 提供多种方式：

```python
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# 方式1：@ 运算符（推荐，Python 3.5+）
C = A @ B
# [[19 22]
#  [43 50]]

# 方式2：np.dot
C = np.dot(A, B)

# 方式3：np.matmul
C = np.matmul(A, B)
```

> 注意：`*` 是逐元素乘法（Hadamard 积），不是矩阵乘法！
> `python A * B # [[5, 12], [21, 32]]（逐元素相乘，不是矩阵乘法）`

6.2.2 矩阵转置

```python
A = np.array([[1, 2, 3], [4, 5, 6]])   # shape (2, 3)

A.T        # 转置，shape (3, 2)
# [[1, 4],
#  [2, 5],
#  [3, 6]]

np.transpose(A)   # 等价于 A.T
```

6.2.3 矩阵的逆

```
A = np.array([[1, 2], [3, 4]])

A_inv = np.linalg.inv(A)   # 逆矩阵
# [[-2. ,  1. ],
#  [ 1.5, -0.5]]

# 验证：A @ A_inv = 单位矩阵
print(A @ A_inv)
# [[1, 0],
#  [0, 1]]（有浮点误差，接近单位矩阵）
```

> 只有方阵（行数=列数）且行列式不为 0 的矩阵才有逆矩阵。奇异矩阵（行列式为0）求逆会报错。

6.2.4 行列式

```
A = np.array([[1, 2], [3, 4]])
np.linalg.det(A)   # -2.0（行列式）
```

6.2.5 特征值与特征向量

```
A = np.array([[2, 1], [1, 2]])
eigenvalues, eigenvectors = np.linalg.eig(A)
print(eigenvalues)     # [3, 1]（特征值）
print(eigenvectors)    # 特征向量矩阵，每列是一个特征向量
```

> 特征值分解在 PCA（主成分分析）、谱聚类等算法中是核心。本项目虽然不直接用，但理解特征值有助于理解降维算法。

6.2.6 解线性方程组

```
# 解 Ax = b
A = np.array([[2, 1], [1, 3]])   # 系数矩阵
b = np.array([5, 10])             # 右端向量

x = np.linalg.solve(A, b)   # 解
print(x)   # [1, 3]（验证：2*1+1*3=5, 1*1+3*3=10）
```

6.2.7 向量内积与外积

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 内积（点积）：标量
np.dot(a, b)    # 32（1*4 + 2*5 + 3*6）
a @ b           # 32（等价）

# 外积：矩阵
np.outer(a, b)
# [[ 4  5  6]
#  [ 8 10 12]
#  [12 15 18]]
```

6.2.8 范数（Norm）

```
a = np.array([3, 4])

np.linalg.norm(a)         # 5.0（L2范数，欧几里得距离）
np.linalg.norm(a, ord=1)  # 7.0（L1范数，曼哈顿距离）
np.linalg.norm(a, ord=np.inf)  # 4.0（无穷范数，最大绝对值）
```

> 本项目中，距离计算本质上就是范数。欧几里得距离 = L2 范数，曼哈顿距离 = L1 范数。但经纬度是球面坐标，不能直接用平面范数，需要用 Haversine 公式。

6.2.9 线性代数在本项目中的应用

本项目不直接大量使用线性代数，但理解以下概念很重要：

1. **距离矩阵**：本质上是矩阵运算，用广播实现；
2. **k-means**：本质上是最小化 SSE（簇内平方和），可以用矩阵形式表达；
3. **标准化**：本质上是矩阵的逐元素运算（减去均值向量，除以标准差向量）；
4. **PCA 降维**（如果数据维度高）：用特征值分解。

6.3 项目实现步骤

1. 练习矩阵乘法、转置、逆、行列式；
2. 练习解线性方程组；
3. 理解范数与距离的关系。

6.4 完整代码（矩阵运算练习）

```
# ============================================================
# 课时6：矩阵运算与线性代数
# ============================================================

import numpy as np

# ------------------------------------------------------------
# 1. 矩阵乘法
# ------------------------------------------------------------
print("=== 1. 矩阵乘法 ===")
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(f"A:\n{A}")
print(f"B:\n{B}")
print(f"A @ B（矩阵乘法）:\n{A @ B}")
print(f"A * B（逐元素乘法，不是矩阵乘法！）:\n{A * B}")

# ------------------------------------------------------------
# 2. 转置、逆、行列式
# ------------------------------------------------------------
print("\n=== 2. 转置、逆、行列式 ===")
C = np.array([[1, 2, 3], [4, 5, 6]])
print(f"C shape: {C.shape}")
print(f"C.T（转置）shape: {C.T.shape}")
print(f"C.T:\n{C.T}")

D = np.array([[1, 2], [3, 4]])
print(f"\nD:\n{D}")
print(f"det(D) = {np.linalg.det(D):.1f}（行列式）")
print(f"D 的逆矩阵:\n{np.linalg.inv(D)}")
print(f"验证 D @ D_inv ≈ I:\n{D @ np.linalg.inv(D)}")

# ------------------------------------------------------------
# 3. 特征值与特征向量
# ------------------------------------------------------------
print("\n=== 3. 特征值与特征向量 ===")
```

```
E = np.array([[2, 1], [1, 2]])
eigenvalues, eigenvectors = np.linalg.eig(E)
print(f"矩阵:\n{E}")
print(f"特征值: {eigenvalues}")
print(f"特征向量（每列一个）:\n{eigenvectors}")

# ------------------------------------------------------------
# 4. 解线性方程组
# ------------------------------------------------------------
print("\n=== 4. 解线性方程组 Ax = b ===")
A = np.array([[2, 1], [1, 3]])
b = np.array([5, 10])
x = np.linalg.solve(A, b)
print(f"A:\n{A}")
print(f"b: {b}")
print(f"解 x: {x}")
print(f"验证 A @ x = {A @ x}（应等于 b）")

# ------------------------------------------------------------
# 5. 向量内积、外积、范数
# ------------------------------------------------------------
print("\n=== 5. 向量内积、外积、范数 ===")
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(f"a = {a}, b = {b}")
print(f"内积 a·b = {np.dot(a, b)}（标量）")
print(f"外积 a⊗b:\n{np.outer(a, b)}（矩阵）")

v = np.array([3, 4])
print(f"\nv = {v}")
print(f"L2范数（欧几里得）: {np.linalg.norm(v)}")
print(f"L1范数（曼哈顿）: {np.linalg.norm(v, ord=1)}")
print(f"无穷范数: {np.linalg.norm(v, ord=np.inf)}")

# ------------------------------------------------------------
# 6. 项目应用：距离与范数
# ------------------------------------------------------------
print("\n=== 6. 项目应用：平面距离 vs 球面距离 ===")
```

```
# 平面上两点距离 = L2范数
p1 = np.array([0, 0])
p2 = np.array([3, 4])
print(f"平面点 p1={p1}, p2={p2}")
print(f"平面距离（L2范数）: {np.linalg.norm(p2 - p1)}")

# 球面上两点距离 = Haversine（不能直接用L2范数，因为经纬度不是平面坐标）
lng1, lat1 = 123.41, 41.80
lng2, lat2 = 123.43, 41.80
# 简化：用经度差*111km（近似，仅演示）
approx_dist = abs(lng2 - lng1) * 111
print(f"\n球面点: ({lng1},{lat1}) → ({lng2},{lat2})")
print(f"近似距离（经度差*111km）: {approx_dist:.2f} km")
print("→ 精确球面距离需要用 Haversine 公式（课时8详解）")
```

6.5 运行结果与解读

运行后展示矩阵运算、线性代数、范数、项目应用。特别注意：
- `@` 是矩阵乘法，`*` 是逐元素乘法，不要混淆；
- L2 范数 = 欧几里得距离，但经纬度是球面坐标，需要用 Haversine；
- 特征值分解是 PCA 等降维算法的核心。

6.6 当堂练习

1. 创建两个 3x3 矩阵 A 和 B，计算 `A @ B`、`A * B`、`A.T`，观察区别；
2. 解线性方程组 `[[1,2],[3,4]] @ x = [5, 11]`，验证解的正确性；
3. 思考：为什么经纬度距离不能直接用 L2 范数？（提示：经纬度是角度，不是平面坐标，1度经度和1度纬度对应的实际距离不同）

6.7 课时小结

- 矩阵乘法用 `@`，逐元素乘法用 `*`；
- 转置 `.T`、逆 `np.linalg.inv`、行列式 `np.linalg.det`；
- 特征值分解 `np.linalg.eig`，解方程组 `np.linalg.solve`；
- 范数：L2=欧几里得，L1=曼哈顿；
- 经纬度是球面坐标，距离用 Haversine 而非平面范数。

课时7：性能对比实验

7.1 课时目标

通过系统性的性能对比实验，深入理解 Python 循环 vs NumPy 向量化 vs Pandas apply 的性能差异。

7.2 知识点深度讲解

7.2.1 三种计算方式的本质

1. **Python 循环**：逐行解释，每次迭代有类型检查、函数调用、对象创建开销；
2. **NumPy 向量化**：底层 C 实现，直接操作连续内存，利用 SIMD 指令；
3. **Pandas apply**：本质上还是 Python 循环（逐行调用函数），但有一些优化。

性能排序：NumPy 向量化 >> Pandas apply > Python 循环。

7.2.2 什么时候用哪种

- **能用 NumPy 向量化就用**：速度最快，代码简洁；
- **复杂逻辑无法向量化时用 Pandas apply**：比纯 Python 循环方便，但仍较慢；
- **绝对避免纯 Python 循环处理大数据**：除非数据量很小（<1000行）。

7.2.3 向量化的常见模式

1. **逐元素运算**：`a + b`、`np.sin(a)`、`a > 5`；
2. **广播运算**：不同 shape 数组运算；
3. **布尔索引**：`a[a > 5]`；
4. **聚合运算**：`np.sum(a, axis=0)`；
5. **矩阵运算**：`A @ B`。

> 本项目中，几乎所有计算都可以用上述模式向量化。如果发现自己在写 for 循环处理数组，先停下来想想能不能用向量化替代。

7.3 项目实现步骤

1. 设计三个性能实验：逐元素运算、距离计算、条件过滤；
2. 分别用 Python 循环、Pandas apply、NumPy 向量化实现；
3. 计时对比，分析差异原因。

7.4 完整代码（性能对比实验）

```
# ============================================================
# 课时7：性能对比实验
# 功能：Python循环 vs Pandas apply vs NumPy向量化
# ============================================================

import time
import numpy as np
import pandas as pd

N = 100000   # 10万条数据

# 生成测试数据
np.random.seed(42)
lons = np.random.uniform(123.3, 123.5, N)
lats = np.random.uniform(41.7, 41.9, N)
df = pd.DataFrame({"lon": lons, "lat": lats})

print("=" * 60)
print(f"性能对比实验（数据量：{N} 条）")
print("=" * 60)

# ------------------------------------------------------------
# 实验1：逐元素运算（计算 sin(lon) * cos(lat)）
# ------------------------------------------------------------
print("\n--- 实验1：逐元素运算 sin(lon)*cos(lat) ---")

# 方式1：Python 循环
t0 = time.time()
result_py = []
for i in range(N):
    result_py.append(np.sin(lons[i]) * np.cos(lats[i]))
t_py = time.time() - t0
print(f"Python 循环：{t_py:.4f} 秒")

# 方式2：Pandas apply
t0 = time.time()
```

```
result_apply = df.apply(lambda r: np.sin(r["lon"]) * np.cos(r["lat"]), axis=1)
t_apply = time.time() - t0
print(f"Pandas apply: {t_apply:.4f} 秒")

# 方式3：NumPy 向量化
t0 = time.time()
result_np = np.sin(lons) * np.cos(lats)
t_np = time.time() - t0
print(f"NumPy 向量化: {t_np:.6f} 秒")

print(f"→ NumPy 比 Python 循环快 {t_py/t_np:.0f} 倍")
print(f"→ NumPy 比 Pandas apply 快 {t_apply/t_np:.0f} 倍")

# ------------------------------------------------------------
# 实验2：距离计算（每个点到固定点的球面距离）
# ------------------------------------------------------------
print("\n--- 实验2：距离计算（每个点到(123.4, 41.8)的球面距离） ---")

center_lon, center_lat = 123.4, 41.8

def haversine_py(lon1, lat1, lon2, lat2):
    """Python 版本 Haversine（单个点）"""
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    a = np.sin((lat2-lat1)/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin((lon2-lon1)/2)**2
    return 6371.0 * 2 * np.arcsin(np.sqrt(a))

# 方式1：Python 循环
t0 = time.time()
dist_py = []
for i in range(N):
    dist_py.append(haversine_py(lons[i], lats[i], center_lon, center_lat))
t_py = time.time() - t0
print(f"Python 循环: {t_py:.4f} 秒")

# 方式2：Pandas apply
t0 = time.time()
dist_apply = df.apply(lambda r: haversine_py(r["lon"], r["lat"], center_lon,
center_lat), axis=1)
```

```
t_apply = time.time() - t0
print(f"Pandas apply: {t_apply:.4f} 秒")

# 方式3：NumPy 向量化
t0 = time.time()
lon1, lat1 = np.radians(lons), np.radians(lats)
lon2, lat2 = np.radians(center_lon), np.radians(center_lat)
a = np.sin((lat2-lat1)/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin((lon2-lon1)/2)**2
dist_np = 6371.0 * 2 * np.arcsin(np.sqrt(a))
t_np = time.time() - t0
print(f"NumPy 向量化: {t_np:.6f} 秒")

print(f"→ NumPy 比 Python 循环快 {t_py/t_np:.0f} 倍")
print(f"→ NumPy 比 Pandas apply 快 {t_apply/t_np:.0f} 倍")

# ------------------------------------------------------------
# 实验3：条件过滤（距离<5km的点）
# ------------------------------------------------------------
print("\n--- 实验3：条件过滤（距离<5km的点） ---")

# 方式1：Python 循环
t0 = time.time()
filtered_py = []
for i in range(N):
    if dist_py[i] < 5:
        filtered_py.append(i)
t_py = time.time() - t0
print(f"Python 循环: {t_py:.4f} 秒, 过滤出 {len(filtered_py)} 个点")

# 方式2：Pandas 布尔索引
t0 = time.time()
filtered_pd = df[dist_apply < 5]
t_pd = time.time() - t0
print(f"Pandas 布尔索引: {t_pd:.6f} 秒, 过滤出 {len(filtered_pd)} 个点")

# 方式3：NumPy 布尔索引
t0 = time.time()
filtered_np = dist_np < 5
```

```
t_np = time.time() - t0
print(f"NumPy 布尔索引：{t_np:.6f} 秒，过滤出 {filtered_np.sum()} 个点")


print(f"→ NumPy 比 Python 循环快 {t_py/t_np:.0f} 倍")


# ------------------------------------------------------------
# 总结
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("总结：")
print("  1. NumPy 向量化最快（C实现，连续内存，SIMD）")
print("  2. Pandas apply 次之（本质还是Python循环）")
print("  3. Python 循环最慢（逐行解释，类型检查开销大）")
print("  4. 数据量越大，NumPy 优势越明显")
print("  5. 本项目53万行数据，必须用NumPy向量化")
print("=" * 60)
```

7.5 运行结果与解读

运行后看到三个实验的性能对比：
- 逐元素运算：NumPy 比 Python 循环快几十到上百倍；
- 距离计算：NumPy 比 Python 循环快几十倍；
- 条件过滤：NumPy 布尔索引最快。

> 具体加速比因电脑性能而异，但趋势一致。关键结论：**能用 NumPy 向量化就不要用 Python 循环。**

7.6 当堂练习

1. 把 N 改成 100 万，重新运行实验1，观察加速比变化；
2. 用 Python 循环实现两个 1000 维向量的内积，再用 NumPy `np.dot` 实现，对比速度；
3. 思考：为什么 Pandas apply 比纯 Python 循环快一些，但远不如 NumPy 向量化？（提示：apply 底层还是循环，但有一些 C 级别的优化）

7.7 课时小结

- 性能排序：NumPy 向量化 >> Pandas apply > Python 循环；
- NumPy 快的原因：C 实现、连续内存、SIMD 指令、无类型检查开销；

- 能用向量化就不要用循环；
- 本项目 53 万行数据必须用向量化，否则太慢。

课时8：项目实战——Haversine 距离向量化

8.1 课时目标

用 NumPy 向量化实现 Haversine 球面距离公式，计算 10 万点的距离矩阵，这是本项目最核心的基础函数。

8.2 知识点深度讲解

8.2.1 为什么需要 Haversine

经纬度是球面坐标（角度），不是平面坐标。直接用欧几里得距离（L2 范数）计算经纬度差是错误的，因为：
1. 经度 1 度和纬度 1 度对应的实际距离不同（纬度越高，经度 1 度越短）；
2. 球面是曲面，两点之间的最短路径是大圆（Great Circle）的弧长，不是直线。

Haversine 公式是计算球面上两点间距离的标准公式。

8.2.2 Haversine 公式推导

设两点经纬度为 (ϕ_1, λ_1) 和 (ϕ_2, λ_2)（ϕ 是纬度，λ 是经度），地球半径为 R。

Haversine 公式：

$$
a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)
$$

$$
c = 2 \cdot \arctan2\left(\sqrt{a}, \sqrt{1-a}\right)
$$

$$
d = R \cdot c
$$

其中：
- $\Delta\phi = \phi_2 - \phi_1$（纬度差）

- $\Delta\lambda = \lambda_2 - \lambda_1$（经度差）
- $R = 6371$ km（地球平均半径）

简化版（用 arcsin 代替 arctan2）：

$$
d = 2R \cdot \arcsin\left(\sqrt{a}\right)
$$

> 本项目用简化版，因为 a 的范围在 [0, 1] 内，arcsin 足够精确。

8.2.3 向量化实现要点

1. 角度转弧度：`np.radians()`（或 `* np.pi / 180`）；
2. 所有运算都是逐元素的，直接用 NumPy 函数；
3. 支持标量和数组混合（广播机制）；
4. 距离矩阵用广播实现（`X[:, None, :] - centers[None, :, :]`）。

8.2.4 本项目中 Haversine 的应用场景

1. **阶段1**：连续段内相邻点距离累加；
2. **阶段3**：每个点到簇中心的距离；
3. **出行目的**：驻留段中心到目的点的距离；
4. **出行方式**：出行段点到地铁站/公交站的距离；
5. **实时热力**：流点到商圈中心的距离。

> Haversine 是本项目使用最频繁的函数，必须向量化实现，否则性能不可接受。

8.3 项目实现步骤

1. 实现向量化的 Haversine 函数；
2. 验证单点距离（与已知距离对比）；
3. 计算 10 万点到固定点的距离；
4. 计算 1000x1000 距离矩阵（广播实现）。

8.4 完整代码（Haversine 向量化实现）

```
# ============================================================
# 课时8：项目实战——Haversine 距离向量化
# 功能：实现球面距离公式，计算单点距离、批量距离、距离矩阵
# ============================================================

import numpy as np
import time

# ------------------------------------------------------------
# 1. 向量化 Haversine 函数
# ------------------------------------------------------------
def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """计算两点间的球面距离（Haversine公式，向量化实现）。

    支持标量和数组输入（利用广播机制）。

    Args:
        lon1, lat1: 第一点经纬度（标量或数组）
        lon2, lat2: 第二点经纬度（标量或数组）
        unit: "km"（千米）或 "m"（米）

    Returns:
        距离（km或m），标量或数组（与输入shape一致）

    公式：
        a = sin²(Δφ/2) + cos(φ1)·cos(φ2)·sin²(Δλ/2)
        d = 2R·arcsin(√a)
    """
    # 地球半径
    R = 6371.0   # km

    # 角度转弧度（np.radians 支持数组）
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])

    # 经纬度差
    dlat = lat2 - lat1
```

```
    dlon = lon2 - lon1

    # Haversine 公式
    a = np.sin(dlat / 2) ** 2 + \
        np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c

    # 单位转换
    return d * 1000 if unit == "m" else d


# ------------------------------------------------------------
# 2. 验证：单点距离
# ------------------------------------------------------------
print("=" * 60)
print("验证：单点距离")
print("=" * 60)

# 沈阳太原街 → 沈阳中街（已知距离约 2km）
taiyuanjie = (123.4119, 41.8052)
zhongjie = (123.4370, 41.8033)
d = haversine(taiyuanjie[0], taiyuanjie[1], zhongjie[0], zhongjie[1])
print(f"太原街 ({taiyuanjie[0]}, {taiyuanjie[1]}) → 中街 ({zhongjie[0]},
{zhongjie[1]})")
print(f"距离：{d:.3f} km（实际约 2km，验证通过）")

# 同一点距离应为 0
d0 = haversine(123.4, 41.8, 123.4, 41.8)
print(f"同一点距离：{d0:.6f} km（应为0）")

# ------------------------------------------------------------
# 3. 批量距离：10万点到固定点
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("批量距离：10万点到固定点")
print("=" * 60)
```

```
N = 100000
np.random.seed(42)
lons = np.random.uniform(123.3, 123.5, N)
lats = np.random.uniform(41.7, 41.9, N)

t0 = time.time()
distances = haversine(lons, lats, 123.4, 41.8)   # 向量化，一次算10万点
t_np = time.time() - t0

print(f"计算 {N} 个点到 (123.4, 41.8) 的距离")
print(f"耗时: {t_np:.4f} 秒")
print(f"距离范围: [{distances.min():.3f}, {distances.max():.3f}] km")
print(f"平均距离: {distances.mean():.3f} km")
print(f"距离<5km的点数: {(distances < 5).sum()}")

# ------------------------------------------------------------
# 4. 距离矩阵：1000x1000（广播实现）
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("距离矩阵：1000x1000（广播实现）")
print("=" * 60)

M = 1000
points = np.column_stack([
    np.random.uniform(123.3, 123.5, M),
    np.random.uniform(41.7, 41.9, M)
])   # shape (1000, 2)

# 广播实现距离矩阵
# points[:, None, :] → (1000, 1, 2)
# points[None, :, :] → (1, 1000, 2)
# 广播后 → (1000, 1000, 2)
t0 = time.time()
lon1 = points[:, 0][:, None]   # (1000, 1)
lat1 = points[:, 1][:, None]   # (1000, 1)
lon2 = points[:, 0][None, :]   # (1, 1000)
lat2 = points[:, 1][None, :]   # (1, 1000)
dist_matrix = haversine(lon1, lat1, lon2, lat2)   # (1000, 1000)
```

```
t_matrix = time.time() - t0

print(f"距离矩阵 shape: {dist_matrix.shape}")
print(f"耗时: {t_matrix:.4f} 秒")
print(f"对角线（自身距离）应为0: {np.allclose(np.diag(dist_matrix), 0)}")
print(f"矩阵对称: {np.allclose(dist_matrix, dist_matrix.T)}")
print(f"距离范围: [{dist_matrix.min():.3f}, {dist_matrix.max():.3f}] km")

# ------------------------------------------------------------
# 5. 对比：双重循环实现距离矩阵（慢，只算100x100）
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("对比：双重循环 vs 广播（100x100）")
print("=" * 60)

S = 100
small_points = points[:S]

# 双重循环
t0 = time.time()
dist_loop = np.zeros((S, S))
for i in range(S):
    for j in range(S):
        dist_loop[i, j] = haversine(small_points[i, 0], small_points[i, 1],
                                    small_points[j, 0], small_points[j, 1])
t_loop = time.time() - t0

# 广播
t0 = time.time()
lon1 = small_points[:, 0][:, None]
lat1 = small_points[:, 1][:, None]
lon2 = small_points[:, 0][None, :]
lat2 = small_points[:, 1][None, :]
dist_broad = haversine(lon1, lat1, lon2, lat2)
t_broad = time.time() - t0

print(f"双重循环: {t_loop:.4f} 秒")
print(f"广播实现: {t_broad:.6f} 秒")
```

```
print(f"加速比：{t_loop / t_broad:.0f} 倍")
print(f"结果一致：{np.allclose(dist_loop, dist_broad)}")

print("\n" + "=" * 60)
print("结论：Haversine 向量化实现是本项目的核心基础函数")
print("所有距离计算都用这个函数，性能有保障")
print("=" * 60)
```

8.5 运行结果与解读

运行后看到：
- 单点距离验证：太原街→中街约 2km，与实际一致；
- 10 万点批量距离：零点几秒完成；
- 1000x1000 距离矩阵：用广播实现，几秒完成；
- 双重循环 vs 广播：100x100 就有几十倍加速比，1000x1000 会有上千倍。

> 这个 Haversine 函数会被抽到 `project_code/utils.py`，供所有模块 import 使用。

8.6 当堂练习

1. 用 Haversine 函数计算北京(116.4, 39.9)到上海(121.5, 31.2)的距离，验证是否约 1067km；
2. 计算 1 万个点中，距离中心点(123.4, 41.8)最近的 10 个点的索引（提示：用 `np.argsort`）；
3. 思考：为什么距离矩阵是对称的？为什么对角线是 0？（提示：距离的对称性和自反性）

8.7 课时小结

- Haversine 公式计算球面距离，是本项目最核心的基础函数；
- 向量化实现支持标量和数组，利用广播机制；
- 距离矩阵用广播实现，比双重循环快成百上千倍；
- 这个函数会被抽到 `utils.py`，供所有模块复用。

三、本日小结

1. **NumPy 设计思想**：ndarray 内存连续、类型固定、向量化运算快；
2. **ndarray 创建与属性**：多种创建方式、shape/dtype/size 等核心属性；
3. **索引与切片**：一维/二维索引、布尔索引、花式索引、视图 vs 副本；

4. **广播机制**：从右往左比较维度，相同或其中一个为 1 则兼容；
5. **数学与统计函数**：逐元素数学函数、聚合统计函数、axis 参数、keepdims；
6. **矩阵运算与线性代数**：矩阵乘法 `@`、转置、逆、特征值、范数；
7. **性能对比**：NumPy 向量化 >> Pandas apply > Python 循环；
8. **Haversine 向量化**：球面距离公式，本项目最核心的基础函数。

四、本日作业

1. 用 NumPy 实现一个函数，输入一组经纬度，返回每个点到所有其他点的距离矩阵（用广播）；
2. 对比 `np.sum(a, axis=0)` 和 `np.sum(a, axis=1)` 的结果，用自己的话解释 axis 的含义；
3. 用 Haversine 函数计算清洗后数据中，每个用户的相邻点距离分布，画直方图；
4. 预习 Day03：思考"Pandas 的 DataFrame 和 NumPy 的 ndarray 有什么区别和联系"。